

Protocol Buffers Documentation

Overview

Protocol Buffers are a language-neutral, platform-neutral extensible mechanism for serializing structured data.

It's like JSON, except it's smaller and faster, and it generates native language bindings. You define how you want your data to be structured once, then you can use special generated source code to easily write and read your structured data to and from a variety of data streams and using a variety of languages.

Protocol buffers are a combination of the definition language (created in `.proto` files), the code that the proto compiler generates to interface with data, language-specific runtime libraries, the serialization format for data that is written to a file (or sent across a network connection), and the serialized data.

What Problems do Protocol Buffers Solve?

Protocol buffers provide a serialization format for packets of typed, structured data that are up to a few megabytes in size. The format is suitable for both ephemeral network traffic and long-term data storage. Protocol buffers can be extended with new information without invalidating existing data or requiring code to be updated.

Protocol buffers are the most commonly-used data format at Google. They are used extensively in inter-server communications as well as for archival storage of data on disk. Protocol buffer *messages* and *services* are described by engineer-authored `.proto` files. The following shows an example `message` :

```
message Person {  
  optional string name = 1;  
  optional int32 id = 2;  
  optional string email = 3;  
}
```

The proto compiler is invoked at build time on `.proto` files to generate code in various programming languages (covered in [Cross-language Compatibility](#) later in this topic) to manipulate the corresponding protocol buffer. Each generated class contains simple

accessors for each field and methods to serialize and parse the whole structure to and from raw bytes. The following shows you an example that uses those generated methods:

```
Person john = Person.newBuilder()  
    .setId(1234)  
    .setName("John Doe")  
    .setEmail("jdoe@example.com")  
    .build();  
output = new FileOutputStream(args[0]);  
john.writeTo(output);
```

Because protocol buffers are used extensively across all manner of services at Google and data within them may persist for some time, maintaining backwards compatibility is crucial. Protocol buffers allow for the seamless support of changes, including the addition of new fields and the deletion of existing fields, to any protocol buffer without breaking existing services. For more on this topic, see [Updating Proto Definitions Without Updating Code](#), later in this topic.

What are the Benefits of Using Protocol Buffers?

Protocol buffers are ideal for any situation in which you need to serialize structured, record-like, typed data in a language-neutral, platform-neutral, extensible manner. They are most often used for defining communications protocols (together with gRPC) and for data storage.

Some of the advantages of using protocol buffers include:

- Compact data storage
- Fast parsing
- Availability in many programming languages
- Optimized functionality through automatically-generated classes

Cross-language Compatibility

The same messages can be read by code written in any supported programming language. You can have a Java program on one platform capture data from one software system, serialize it based on a `.proto` definition, and then extract specific values from that serialized data in a separate Python application running on another platform.

The following languages are supported directly in the protocol buffers compiler, protoc:

- [C++](#)

- [C#](#)
- [Java](#)
- [Kotlin](#)
- [Objective-C](#)
- [PHP](#)
- [Python](#)
- [Ruby](#)

The following languages are supported by Google, but the projects' source code resides in GitHub repositories. The protoc compiler uses plugins for these languages:

- [Dart](#)
- [Go](#)

Additional languages are not directly supported by Google, but rather by other GitHub projects. These languages are covered in [Third-Party Add-ons for Protocol Buffers](#).

Cross-project Support

You can use protocol buffers across projects by defining `message` types in `.proto` files that reside outside of a specific project's code base. If you're defining `message` types or enums that you anticipate will be widely used outside of your immediate team, you can put them in their own file with no dependencies.

A couple of examples of proto definitions widely-used within Google are [timestamp.proto](#) and [status.proto](#).

Updating Proto Definitions Without Updating Code

It's standard for software products to be backward compatible, but it is less common for them to be forward compatible. As long as you follow some [simple practices](#) when updating `.proto` definitions, old code will read new messages without issues, ignoring any newly added fields. To the old code, fields that were deleted will have their default value, and deleted repeated fields will be empty. For information on what "repeated" fields are, see [Protocol Buffers Definition Syntax](#) later in this topic.

New code will also transparently read old messages. New fields will not be present in old messages; in these cases protocol buffers provide a reasonable default value.

When are Protocol Buffers not a Good Fit?

Protocol buffers do not fit all data. In particular:

- Protocol buffers tend to assume that entire messages can be loaded into memory at once and are not larger than an object graph. For data that exceeds a few megabytes, consider a different solution; when working with larger data, you may effectively end up with several copies of the data due to serialized copies, which can cause surprising spikes in memory usage.
- When protocol buffers are serialized, the same data can have many different binary serializations. You cannot compare two messages for equality without fully parsing them.
- Messages are not compressed. While messages can be zipped or gzipped like any other file, special-purpose compression algorithms like the ones used by JPEG and PNG will produce much smaller files for data of the appropriate type.
- Protocol buffer messages are less than maximally efficient in both size and speed for many scientific and engineering uses that involve large, multi-dimensional arrays of floating point numbers. For these applications, [FITS](#) and similar formats have less overhead.
- Protocol buffers are not well supported in non-object-oriented languages popular in scientific computing, such as Fortran and IDL.
- Protocol buffer messages don't inherently self-describe their data, but they have a fully reflective schema that you can use to implement self-description. That is, you cannot fully interpret one without access to its corresponding `.proto` file.
- Protocol buffers are not a formal standard of any organization. This makes them unsuitable for use in environments with legal or other requirements to build on top of standards.

Who Uses Protocol Buffers?

Many projects use protocol buffers, including the following:

- [gRPC](#)
- [Google Cloud](#)
- [Envoy Proxy](#)

How do Protocol Buffers Work?

The following diagram shows how you use protocol buffers to work with your data.

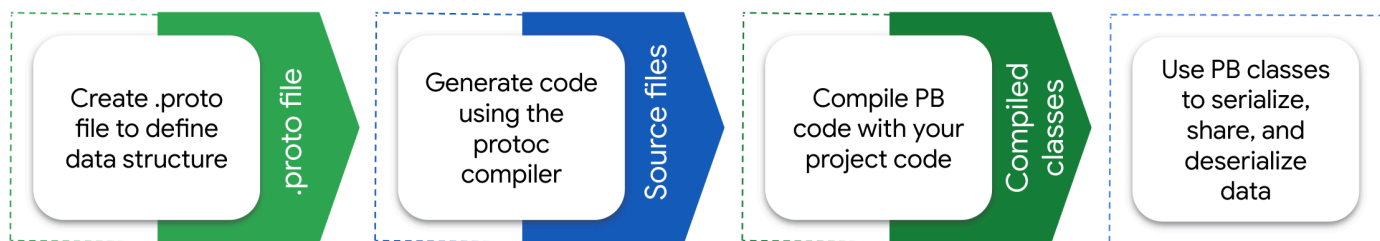


Figure 1. Protocol buffers workflow

The code generated by protocol buffers provides utility methods to retrieve data from files and streams, extract individual values from the data, check if data exists, serialize data back to a file or stream, and other useful functions.

The following code samples show you an example of this flow in Java. As shown earlier, this is a `.proto` definition:

```
message Person {  
  optional string name = 1;  
  optional int32 id = 2;  
  optional string email = 3;  
}
```

Compiling this `.proto` file creates a `Builder` class that you can use to create new instances, as in the following Java code:

```
Person john = Person.newBuilder()  
    .setId(1234)  
    .setName("John Doe")  
    .setEmail("jdoe@example.com")  
    .build();  
output = new FileOutputStream(args[0]);  
john.writeTo(output);
```

You can then deserialize data using the methods protocol buffers creates in other languages, like C++:

```
Person john;  
fstream input(argv[1], ios::in | ios::binary);  
john.ParseFromIstream(&input);  
int id = john.id();  
std::string name = john.name();  
std::string email = john.email();
```

Protocol Buffers Definition Syntax

When defining `.proto` files, you can specify that a field is either `optional` or `repeated` (proto2 and proto3) or leave it set to the default, implicit presence, in proto3. (The option to set a field to `required` is absent in proto3 and strongly discouraged in proto2. For more on this, see [Required Fields Considered Harmful](#).)

After setting the optionality/repeatability of a field, you specify the data type. Protocol buffers support the usual primitive data types, such as integers, booleans, and floats. For the full list, see [Scalar Value Types](#).

A field can also be of:

- A `message` type, so that you can nest parts of the definition, such as for repeating sets of data.
- An `enum` type, so you can specify a set of values to choose from.
- A `oneof` type, which you can use when a message has many optional fields and at most one field will be set at the same time.
- A `map` type, to add key-value pairs to your definition.

In proto2, messages can allow **extensions** to define fields outside of the message, itself. For example, the protobuf library's internal message schema allows extensions for custom, usage-specific options.

For more information about the options available, see the language guide for [proto2](#) or [proto3](#).

After setting optionality and field type, you choose a name for the field. There are some things to keep in mind when setting field names:

- It can sometimes be difficult, or even impossible, to change field names after they've been used in production.
- Field names cannot contain dashes. For more on field name syntax, see [Message and Field Names](#).
- Use pluralized names for repeated fields.

After assigning a name to the field, you assign a field number. Field numbers cannot be repurposed or reused. If you delete a field, you should reserve its field number to prevent someone from accidentally reusing the number.

Additional Data Type Support

Protocol buffers support many scalar value types, including integers that use both variable-length encoding and fixed sizes. You can also create your own composite data types by

defining messages that are, themselves, data types that you can assign to a field. In addition to the simple and composite value types, several [common types](#) are published.

History

To read about the history of the protocol buffers project, see [History of Protocol Buffers](#).

Protocol Buffers Open Source Philosophy

Protocol buffers were open sourced in 2008 as a way to provide developers outside of Google with the same benefits that we derive from them internally. We support the open source community through regular updates to the language as we make those changes to support our internal requirements. While we accept select pull requests from external developers, we cannot always prioritize feature requests and bug fixes that don't conform to Google's specific needs.

Developer Community

To be alerted to upcoming changes in Protocol Buffers and to connect with protobuf developers and users, [join the Google Group](#).

Additional Resources

- [Protocol Buffers GitHub](#)
- [Codelabs](#)